



KamandPrompt

Programming Club IIT Mandi

HACKATHON PROBLEM STATEMENT

FIRST COMMIT

Build a Real-Time Chat Application

CHALLENGE DURATION
24 HOURS

ORGANIZED BY
KAMAND PROMPT | IIT MANDI

Start with a conversation. Build whatever comes next.

01 Challenge Overview

THE CHALLENGE

Challenge: Build a real-time, web-based chat application that allows users to connect and communicate with one another. While sending a message may seem simple, building a reliable chat platform involves authentication, real-time communication, persistent data, and handling multiple users simultaneously.

THE CONTEXT

Context: The goal is to build the foundation of a modern chat application and then extend it into an experience of your choice.

The application could be a simple personal messaging platform, a campus communication system, a community chat application, or something entirely different. The only requirement is that the core communication experience works reliably.

02 Mandatory Core Requirements

01 User Authentication

Implement user registration and login functionality.

- Allow users to create an account.
- Allow registered users to sign in and sign out.
- Maintain an authenticated user session.

02 One-to-One Chat

Allow two registered users to create a private conversation and communicate with each other in real time.

- Users should be able to initiate a conversation with another user.
- Messages should be delivered in real time.
- Messages should be associated with the appropriate users and conversation.

These two components form the **minimum expected functionality** of the application. Teams are expected to build upon this foundation and add features that make their application more complete, useful, or interesting.

03 Build Upon the Core

OPTIONAL EXTENSIONS

Once authentication and one-to-one communication are working, teams are encouraged to expand their application with additional functionality. The following are examples of features that can significantly improve the depth of the application:

- **Group Conversations:** Allow users to create groups, add members, and communicate with multiple users simultaneously.

- **Persistent Database:** Store users, conversations, messages, groups, memberships, timestamps, and other relevant application data in a persistent database.
- **Chat Encryption:** Implement encryption mechanisms to improve the privacy and security of messages.
- **Multimedia Messaging:** Extend communication beyond text by supporting:
 - Images
 - Voice notes
 - File sharing
- **File Safety:** If file sharing is implemented, teams may introduce mechanisms to validate uploaded files and detect potentially malicious or unsafe content.
- **LLM-Powered Conversations:** Integrate a Large Language Model that can participate in conversations and generate responses using relevant chat context.
- **Context-Aware AI:** Teams may explore conversation history, summarization, user preferences, or other approaches to provide more contextually relevant AI responses.
- **Additional Chat Features:** Teams may implement features such as:
 - Message reactions
 - Typing indicators
 - Read receipts
 - Online/offline status
 - Message search
 - Message editing or deletion
 - Notifications
 - User profiles
 - Media previews
 - Chat moderation

The features above are suggestions rather than a fixed checklist. Teams are encouraged to choose the features that best fit their idea and implement them to whatever depth they consider appropriate. Adding more meaningful functionality and polishing the overall experience can significantly improve the quality of the final application.

04 Technical Considerations

- Communication between clients and the server should use **WebSockets** or a **WebSocket-based technology** to enable real-time messaging.
- The mandatory one-to-one chat functionality must work reliably for multiple users.
- Teams should use appropriate authentication and session-management mechanisms.
- Teams are encouraged to use a persistent database as the application grows beyond the minimum requirements.
- A typical architecture may consist of a frontend communicating with a backend responsible for authentication, database operations, and real-time communication.
- Teams may choose any suitable programming language, framework, database, WebSocket implementation, or hosting platform.
- Third-party APIs, libraries, pretrained models, and external services may be used where appropriate.

- API keys, credentials, and other sensitive information must not be exposed in client-side code or committed to the repository.

05 Suggested Technology Stack

The following technologies are suggestions only. Teams are free to use alternatives.

COMPONENT	SUGGESTED TECHNOLOGIES
Frontend	React, Next.js, Vue, Svelte, or other web frameworks.
Backend	Node.js with Express, Python with FastAPI/Flask, Java Spring Boot, or other suitable backend frameworks.
Real-Time Communication	WebSockets, Socket.IO, or other WebSocket-based technologies.
Database	PostgreSQL, MySQL, MongoDB, SQLite, Firebase, Supabase, or other suitable databases.
Authentication	Session-based authentication, JWT, SupaBase or Firebase Authentication, or other suitable mechanisms.
File Storage	Local storage, Firebase Storage, Supabase Storage, cloud object storage, or other suitable solutions.

06 Resources

- **WebSockets Tutorial:**
An introductory resource for understanding WebSocket communication and real-time applications.
<https://www.tutorialspoint.com/websockets/index.htm>
- **WebSocket Chat Application Tutorial:**
A practical video resource demonstrating the implementation of a real-time chat application using WebSockets.
<https://youtu.be/Czcfel7ymbU?si=3kRGRpE0fVUiQI7F>

07 Your Creativity

The core requirements provide only the starting point. Beyond them, the direction of the application is entirely up to you.

Think about what a chat application could become if you were given the freedom to redesign the experience.

You may build features that are not mentioned in this problem statement, combine multiple ideas, or create an entirely new communication experience.

Some possible directions include:

- A campus-specific communication platform
- An AI-assisted messaging application

- A community or interest-based chat platform
- A privacy-focused messaging application
- A multimedia-first communication platform
- A collaborative chat application
- A completely new interaction model for online communication

There is no fixed upper limit on what you can build. Once the mandatory core functionality is complete, the quality, depth, creativity, and usefulness of the features you add are entirely up to you.

08 Evaluation Criteria

The final submission will be evaluated on the following 100-mark rubric:

CRITERION	WHAT IS EVALUATED	WEIGHT
Core Functionality	Authentication, login and logout, user sessions, one-to-one chat, real-time message delivery, and correct association of messages with users.	20%
Technical Implementation	WebSocket implementation, backend architecture, database, authentication, deployment and session handling, reliability, and code quality.	15%
Feature Depth & Usefulness	Meaningful features beyond the mandatory requirements, depth of implementation, usefulness, and integration.	15%
Demo / Working Product	Whether the product works, and the quality, smoothness, reliability, and completeness of the live demonstration.	25%
Pitch / Presentation	Problem-to-solution clarity, uniqueness, product vision, explanation of features and technology, impact, and communication.	25%

STAGE 01 — SHORTLISTING

50 marks total

The first three criteria, worth 50 marks combined, will be used to shortlist teams.

STAGE 02 — FINAL JUDGING

50 additional marks

The final two criteria, worth the remaining 50 marks, will be evaluated only for shortlisted teams. These marks will be combined with the first-stage scores for the final result.

09 Submission Requirements

01 Source Code and README

- Submit the complete source code through a GitHub repository.
- Include a clear README with dependency installation, configuration, setup, and local run instructions.
- The README should explain the application architecture, real-time communication technology, major features, and any required environment variables or external services.
- Include instructions for reproducing and testing the submitted application in a clean environment.
- Teams should keep their GitHub repository private until the end of the hackathon and may make it public when submitting their final entry.

02 Working Web Application

- Submit a working web application demonstrating the mandatory real-time chat functionality.
- The organizing team may run the code and test the application independently.

03 Presentation

- Submit a PPT presentation explaining the idea, implementation, main features, and final product.
- Upload the PPT to Google Drive with public link access enabled and include the shareable link in the submission.

04 Demo Video

- Submit a short video demonstrating the working application and its key features.
- Upload the demo video to YouTube with public access and include the video link in the submission.

Teams may also deploy their application and provide a live link using Vercel, Netlify, Render, Railway, or another suitable hosting platform. Deployment is optional, but a live application may make the final demo easier to showcase.

Important: The YouTube video and Google Drive presentation links must be publicly accessible to the organizers. The source repository should remain private until the hackathon ends and may be made public at submission time. Failure to follow these repository, upload, link-sharing, or public-access requirements will result in disqualification.

TIME LIMIT: 24 HOURS

Start with a conversation. Build whatever comes next.